

To run the Kubernetes cluster, follow the steps given below:

```
$ minikube start
```

Once the cluster has been generated, its status can be confirmed by entering:

```
$ minikube status
```

Setting up a CI/CD pipeline with Kubernetes

Before setting up the pipeline, you should be familiar with Kubernetes, which is an open source container orchestration tool. Its main function is to direct containerised applications on clusters of nodes by serving operators; organise, scale, update, and maintain their services; and provide mechanisms for service discovery.

Setting up/installing Jenkins on Kubernetes

First, we need to install Helm, which is the package manager for Kubernetes:

```
$ curl https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3 > get_helm.sh
$ chmod 700 get_helm.sh
$ ./get_helm.sh -v v3.5.2
```

After that, we have to configure Helm. To do this, add the Jenkins repo, as shown below. We also need to install Tiller for Helm to run correctly:

```
$ kubectl -n kube-system create serviceaccount tiller
Serviceaccount/tiller created
~/.kube
$ kubectl create clusterrolebinding tiller --clusterrole cluster-admin --serviceaccount=kube-system:tiller
clusterrolebinding.rbac.authorization.k8s.io/tiller created
~/.kube
$ helm init --service-account tiller
$HELM_HOME has been configured at /Users/itspare/.helm.
```

Next, we need to run the *inspect* command to verify the configuration values of the deployment:

```
$ helm inspect values stable/jenkins > values.yml
```

Keep a watchful check on the configuration values and make changes if needed. Then install the chart:

```
$ helm install stable/jenkins --tls \
--name jenkins \
--namespace jenkins
```

The installation process will display some instructions for what has to be done next.

Points to remember

Get your 'admin' user password by running:

```
printf $(kubectl get secret --namespace default my-jenkins
-o jsonpath="{.data.jenkins-admin-password}" | base64
--decode);echo
```

Get the Jenkins URL by running these commands in the same shell:

```
export POD_NAME=$(kubectl get pods --namespace default
-l "app.kubernetes.io/component=jenkins-master" -l "app.
kubernetes.io/instance=my-jenkins" -o jsonpath="{.items[0].
metadata.name}")
echo http://127.0.0.1:8080
kubectl --namespace default port-forward $POD_NAME 8080:8080
```

Follow these steps and they will start the proxy server at *http://127.0.0.1:8080*.

Open the above localhost URL, and enter your user name and password that has been created earlier. Your personal Jenkins server will open in a few minutes.

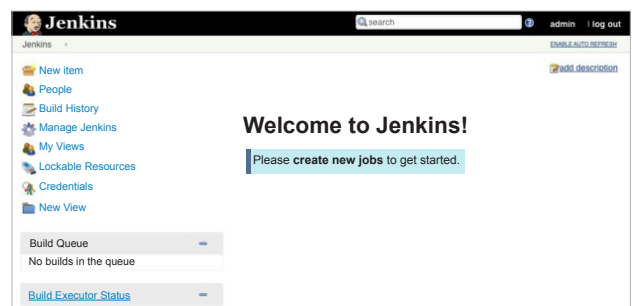


Figure 1: Jenkins home page

Kubernetes and CI/CD practices are an awesome match. We have learnt what a CI/CD pipeline is, why to use it, and how to install it with Kubernetes. Remember what have been installed here are the most basic plugins; there are still lots of configuration options that can be applied. We can also use Kubernetes to scale up the CI/CD pipeline. Hopefully, in the next article we will learn about that. **END** 🐧

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing.